

Week 8: Agile Technical Practices Activity

Duration: 60 minutes

Format: Students in teams (your assignment teams)

Students rotate through several data-analytics mini-scenarios and complete short tasks tied to technical practices. Every task includes built-in discussion generators.

Scenario Overview (All Imaginary Projects)

Students work on three fictional data analytics projects:

Project A: Customer Churn Predictor

A telecom company wants to predict churn using call logs, service usage, and contract length.

Project B: Fraud Detection Pipeline

A financial institution wants anomaly detection on transaction streams, enriched with user metadata.

Project C: Clinical Trial Outcome Analysis

A pharmaceutical company wants to clean, aggregate, and analyze clinical trial measurements with missing data.

Each project is intentionally incomplete or messy to emphasize Agile technical practices.

Part 1: “Building the Right Thing” Analysis (15 min)

Task

Each team can pick one of the projects (A, B, or C). They must list:

1. What the business actually needs
2. What assumptions an engineering team could mistakenly make
3. Questions they would ask stakeholders
4. What the minimum useful analytic output might be

Discussion Prompts

- “Where could analysts build something technically sophisticated but ultimately useless?”
- “Which assumptions are most dangerous?”
- “If this were your team, what would you validate first to avoid waste?”

Part 2: Technical Debt Identification Exercise (15 min)

Each group has a fictional description of the project's current state, for example:

Project A:

The churn pipeline uses four separate scripts that duplicate the same feature engineering logic. Logging is inconsistent. Data validation happens only at the end.

Project B:

The anomaly thresholds are hardcoded. Three different functions compute "risk score" differently. File paths are embedded directly in the code.

Project C:

The data cleaning logic includes 12 nested if-else statements. Variable names are short and ambiguous.

Task

Teams identify:

- 5 sources of technical debt
- The risks associated with each
- Which debts they would attack first and why

Discussion Prompts

- "How does this technical debt slow future analysis?"
- "What small refactoring moves would reduce risk significantly?"
- "Which debt is tolerable vs. harmful?"

Part 3: Refactoring Without Code (15 min)

This is a poorly structured pseudocode snippet representing part of a data pipeline.

Example:

Pseudocode for Project B - Fraud Detection

for each transaction:

 if transaction.amount < 0 or transaction.amount > 50000:

 flag = True

 else:

 flag = False

 if "UNVERIFIED" in transaction.user_status:

 flag = True

 if transaction.country == "X1" or transaction.country == "X2" or transaction.country == "X3":

 flag = True

 if flag:

 risk_score = compute_risk(transaction)

```
if risk_score > 0.8:
    escalate(transaction)
else:
    store(transaction)
else:
    store(transaction)
```

Task

Teams produce a refactored design without writing real code, such as:

- extracting decision rules
- identifying repeated logic
- proposing helper steps
- reorganizing in cleaner blocks

Discussion Prompts

- “How does clearer structure reduce defects?”
- “What naming or modularization decisions mattered most?”
- “How would this improve onboarding a new team member?”

Part 4: Emergent Design & Requirement Change (10 min)

New requirements for each project:

Project A:

Marketing now wants churn reasons grouped by customer segment.

Project B:

Fraud patterns change weekly—thresholds must update dynamically from historical distributions.

Project C:

Clinical trial now includes multi-site data with different measurement units.

Task

Teams explain how the system's design should evolve:

- What changes incrementally?
- What needs redesign?
- What abstractions become necessary?
- What risks do the changes introduce?

Discussion Prompts

- “Which parts of the design were flexible and which were rigid?”

- “What would you defer deciding until you actually have more information?”
- “Which improvements help future adaptability most?”

Part 5: Final Reflection (5 min)

Prompts

- “Which technical practice seemed most important for long-term analytic success?”
- “Where did you notice the connection between design decisions and future change?”
- “Which exercise changed your view of how analytics teams should build pipelines?”

Part 6: TDD Example (30 min)

A Python data pipeline with ingestion, cleaning, and analysis modules is given to you. First run the pytest with the help of your instructor and you will see many tests failing. Complete the TODO or incomplete sections in `ingest.py`, `clean.py`, and `analyze.py`, and finish the test files in `tests/`. Add implementations where needed. All tests should pass when complete. The project just uses the iris dataset from the data folder for representation only. But this should give you an idea of how to write tests in Python.